MOON

Programming Language Specification

Codename: Mars

Version 0.1.0 • Draft Specification

February 2026

CONFIDENTIAL

Dark Matter Tech • Orion Team

Table of Contents

1. Executive Summary
2. Design Philosophy
3. Language Overview
4. Type System
5. Memory Model
6. Concurrency
7. UI Framework (MoonUI)
8. Platform Capability API
9. Hardware Security Bridge
10. Package Manager (Aurora)
11. Standard Library
12. Interoperability
13. Compilation Pipeline
14. Migration Strategy
15. Appendix: Grammar Reference

1. Executive Summary

Moon is a statically-typed, compiled programming language designed to replace JavaScript and the DOM as the foundational technology of the web platform. Developed under codename Mars, Moon targets a new browser runtime that eliminates the legacy constraints of HTML, CSS, and JavaScript while providing native-quality experiences across all platforms.

Moon is not an incremental improvement. It is a ground-up replacement for the web's execution model:

- **Type safety:** All types known at compile time. No `any`, no `dynamic`, no runtime coercion.
- **Memory safety:** ARC with cycle detection. No GC pauses. No manual memory management.
- **Compiled:** AOT compilation for production. Interpreted mode with hot reload for development.
- **No DOM:** Declarative, reactive UI scene graph replaces the Document Object Model entirely.
- **No CSS:** Type-safe styling properties. No cascade, no specificity, no selectors.
- **Platform-aware:** Runtime bridges to native capabilities (Apple Pay, Google Pay, APNs, FCM, biometrics, HSMs).
- **Hardware security:** First-class HSM, secure enclave, and hardware token support.
- **Engine-agnostic:** WebKit on Apple hardware, Blink everywhere else. Developer never cares.

Moon's syntax is inspired by Kotlin. Its memory model borrows from Swift's ARC. Its UI framework draws from SwiftUI, Jetpack Compose, and Flutter's declarative paradigms.

2. Design Philosophy

2.1 Guiding Principles

Declare intent, not mechanics. Developers describe what they want. The runtime handles platform-specific implementation.

One codebase, native everywhere. Moon runs on macOS (WebKit), Android (Blink), Windows (Blink), Linux (Blink) — each delivering its best native experience.

Fail at compile time, not runtime. Type system, null safety, exhaustive matching, and capability declarations catch errors before deployment.

Security is not optional. Hardware-backed crypto, signed packages, capability permissions built in.

Performance is a feature. ARC + AOT + retained-mode renderer = consistent 60/120fps.

2.2 What Moon Replaces

Legacy Technology	Moon Replacement	Why
JavaScript / TypeScript	Moon language	Type-safe, compiled, memory-safe
HTML DOM	MoonUI scene graph	Declarative, reactive, retained-mode
CSS	Inline type-safe styling	No cascade, no specificity wars
npm / node_modules	Aurora package manager	Signed binaries, flat deps
WebAuthn (auth only)	Hardware Crypto Bridge	Full HSM operations
Web Crypto API	Hardware-backed crypto	Keys never leave hardware
Service Workers	Native push integration	APNs/FCM/WNS via bridge
Payment Request API	Native payment bridge	Apple Pay/Google Pay/native

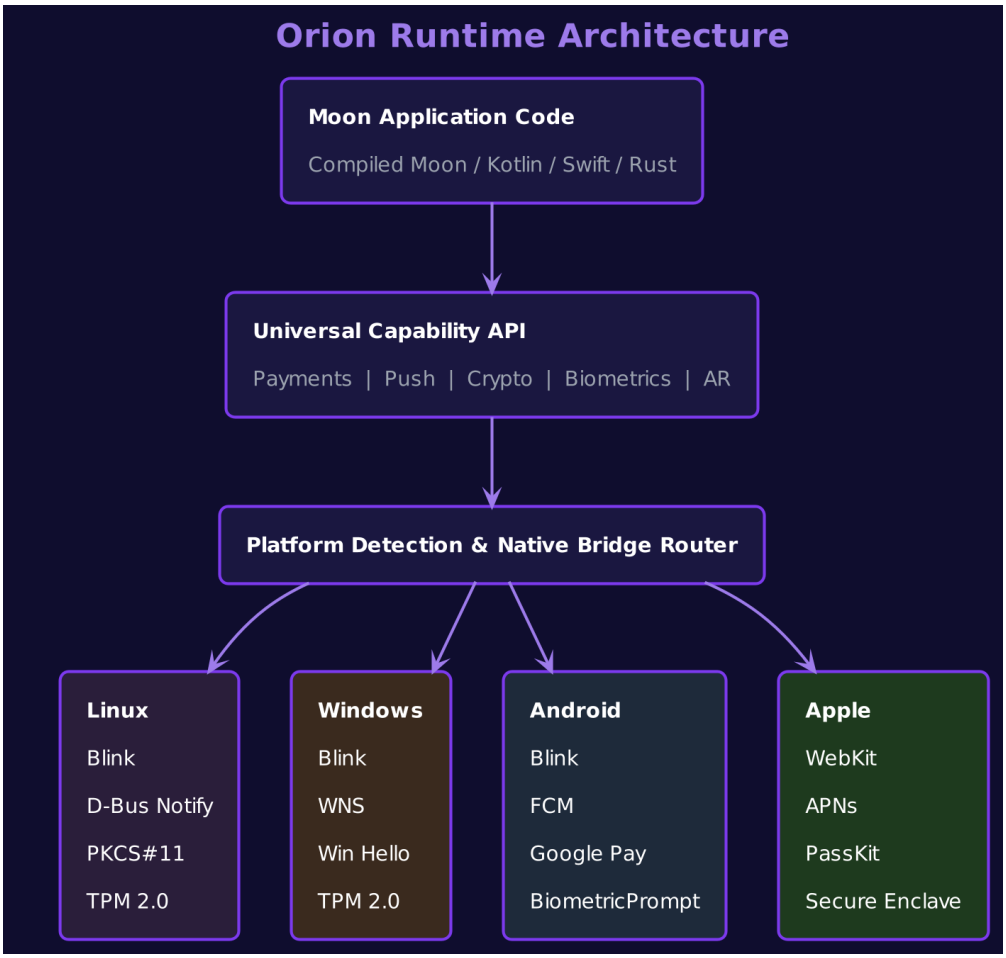

Figure 1: Orion Runtime Architecture

3. Language Overview

Moon uses Kotlin-inspired syntax with enhancements for web platform capabilities. Source files use `.moon`.

3.1 Hello World

```
// main.moon
fun main() {
    print("Hello, Moon!")
}
```

3.2 Variables and Constants

```
val name: String = "Moon"     // immutable
var counter: Int = 0           // mutable
val inferred = 42              // type inference

// Null safety
val required: String = "always set"
val optional: String? = null
val length = optional?.length ?: 0
```

3.3 Functions

```
fun add(a: Int, b: Int): Int {
    return a + b
}

fun multiply(a: Int, b: Int): Int = a * b

fun greet(name: String, greeting: String = "Hello"): String {
    return "${greeting}, ${name}!"
}

val doubled = listOf(1, 2, 3).map { it * 2 }
items.filter { it.price > 10.0 }.sortedBy { it.name }.take(5)
```

3.4 Control Flow

```
val result = when (status) {
    Status.Active  -> "Running"
    Status.Paused  -> "On hold"
    Status.Stopped -> "Done"
}

val label = if (count > 0) "Items: $count" else "Empty"
for (i in 0 until 10) { /* 0..9 */ }
```

3.5 Data Classes

```
data class User(
    val id: String, val name: String,
    val email: String, val role: Role = Role.Viewer
)
val (id, name, email) = user
val admin = user.copy(role = Role.Admin)
```

3.6 Sealed Classes

```
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val message: String) : Result<Nothing>()
    object Loading : Result<Nothing>()
}

fun handle(r: Result<User>) = when (r) {
    is Result.Success -> showUser(r.data)
    is Result.Error   -> showError(r.message)
    Result.Loading    -> showSpinner()
}
```

3.7 Interfaces

```
interface Serializable {
    fun toBytes(): ByteArray
    fun contentType(): String = "application/octet-stream"
}

class ApiResponse(
    override val cacheKey: String, val body: ByteArray
) : Serializable, Cacheable {
    override fun toBytes() = body
}
```

4. Type System

Every value has a known type at compile time. No any type, no implicit coercion, no escape hatches.

4.1 Primitive Types

Type	Size	Description
Int	64-bit	Signed integer (default)
Int8, Int16, Int32	8/16/32	Sized integers
Float32 / Float64	32/64	IEEE 754 floats
Bool	1-bit	true or false
Char	32-bit	Unicode scalar
String	Variable	UTF-8, immutable
Unit / Nothing	0-bit	Void / bottom type

4.2 Null Safety

```
val name: String = "Moon"       // never null
val alias: String? = null        // nullable
val len = alias?.length ?: 0     // safe call + elvis

if (alias != null) {
    print(alias.length)  // smart cast to String
}
```

4.3 Generics

```
class Repository<T : Entity> {
    fun add(item: T) { items.add(item) }
    fun find(id: String): T? = items.firstOrNull { it.id == id }
}

interface Producer<out T> { fun produce(): T }
interface Consumer<in T> { fun consume(item: T) }
```

4.4 Extensions

```
fun String.isValidEmail(): Bool =
    this.matches(Regex("[a-zA-Z0-9+._-]+@[a-zA-Z0-9.-]+"))

val List<Int>.median: Float64
    get() {
        val s = this.sorted()
        return if (size % 2 == 0) (s[size/2-1] + s[size/2]) / 2.0
        else s[size/2].toFloat64()
    }
```

5. Memory Model

Moon uses ARC supplemented by a lightweight cycle detector. Deterministic deallocation, no GC pauses, smooth 60/120fps rendering.

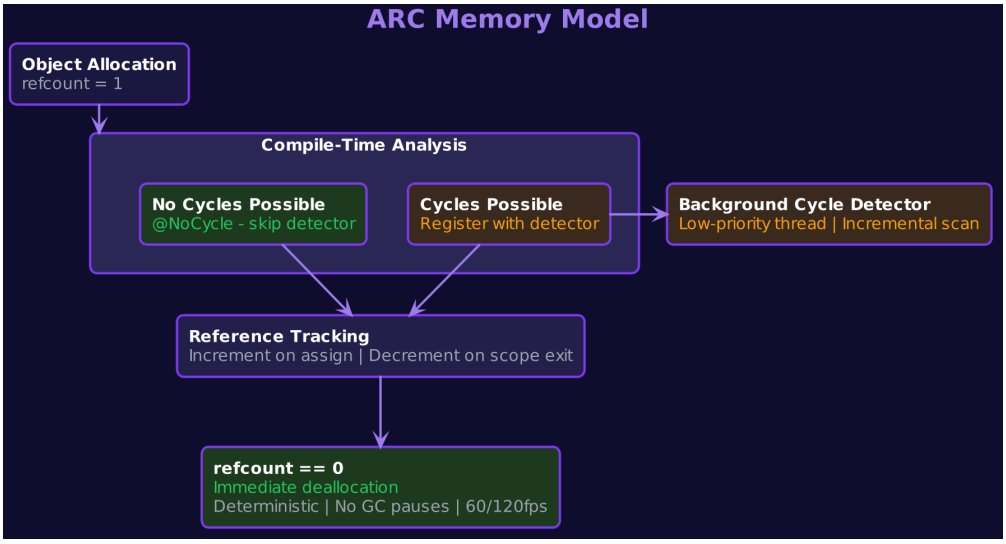

Figure 2: ARC Memory Model

5.1 ARC Semantics

```
fun processOrder(cart: Cart) {
    val order = Order(cart)       // refcount = 1
    val receipt = order.receipt()  // refcount = 2
    sendEmail(receipt)
    // scope exit: refcount 0 -> immediate dealloc
}
```

5.2 Weak and Unowned References

```
class Parent { var children: List<Child> = emptyList() }
class Child { weak var parent: Parent? = null }
class Engine { unowned val vehicle: Vehicle }
```

5.3 Cycle Detector

Compiler marks objects with no possible cycles as `@NoCycle`. Only potential-cycle objects register with a background incremental detector running on a low-priority thread.

```
// Compiler-inferred: no cycle possible
@NoCycle
class Logger(val tag: String)

// Potential cycle: parent <-> child
class TreeNode(val value: String) {
    var children: List<TreeNode> = emptyList()
    weak var parent: TreeNode? = null
}

// Detector runs incrementally, never pauses the UI thread
// Typical cycle collection: < 0.1ms per scan
```

The cycle detector uses a mark-and-sweep variant that runs in incremental slices during idle time. Objects annotated with `@NoCycle` bypass registration entirely, keeping the common case at zero overhead.

6. Concurrency

Structured concurrency inspired by Kotlin coroutines and Swift async/await, with compile-time data race safety.

6.1 Async/Await

```
suspend fun fetchUser(id: String): User {
    val resp = http.get("https://api.example.com/users/$id")
    return resp.decode<User>()
}
```

6.2 Structured Concurrency

```
suspend fun loadDashboard() {
    val (user, orders, notifs) = concurrent {
        val u = async { fetchUser(userId) }
        val o = async { fetchOrders(userId) }
        val n = async { fetchNotifications(userId) }
        Triple(u.await(), o.await(), n.await())
    }
    render(Dashboard(user, orders, notifs))
}
```

6.3 Actors

```
actor ShoppingCart {
    private var items = mutableListOf<CartItem>()
    fun add(item: CartItem) { items.add(item) }
    fun total(): Float64 = items.sumOf { it.price * it.quantity }
}

// All access automatically serialized
cart.add(item)  // safe from any coroutine
```

6.4 Reactive Streams

```
val prices: Flow<Float64> = stockTicker("GOOG")
prices.filter { it > 150.0 }
    .map { PriceAlert("GOOG", it) }
    .collect { notify(it) }

val cartItems = MutableStateFlow<List<CartItem>>(emptyList())
cartItems.value = cartItems.value + newItem  // triggers re-render
```

7. UI Framework (MoonUI)

MoonUI replaces HTML, CSS, and the DOM with a declarative, reactive scene graph. Retained-mode rendering with automatic diffing.

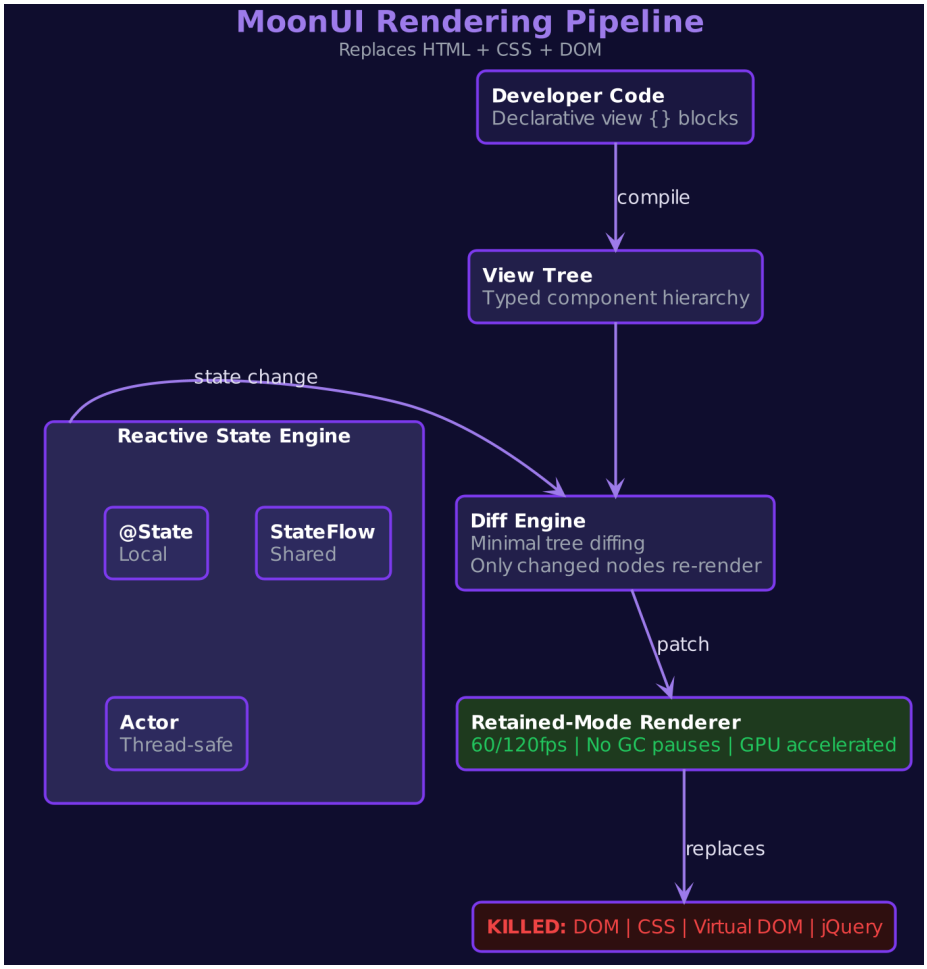

Figure 3: MoonUI Rendering Pipeline

7.1 Declarative Views

```
view ProductCard(product: Product) {
    Column(spacing = 12.dp, padding = 16.dp) {
        Image(product.thumbnailUrl)
            .size(200.dp, 200.dp).cornerRadius(8.dp)
        Text(product.name).font(.headline).color(.primary)
        Text("$${product.price}").font(.body).color(.secondary)
        Button("Add to Cart") { cart.add(product) }
            .style(.primary).fullWidth()
    }
}
```

7.2 State Management

```
view Counter() {
    @State var count = 0
    Column(alignment = .center, spacing = 16.dp) {
        Text("Count: $count").font(.display)
        Row(spacing = 8.dp) {
            Button("-") { count-- }.style(.outlined)
            Button("+") { count++ }.style(.primary)
        }
    }
}
```

7.3 Navigation

```
navigation App {
    route("/") { HomePage() }
    route("/products/{id}") { p -> ProductDetail(p["id"]) }
    route("/cart") { CartPage() }
}
Navigator.push("/products/${product.id}")
```

7.4 Styling (No CSS)

```
val cardStyle = Style {
    background(.surface)
    cornerRadius(12.dp)
    shadow(elevation = 4.dp, color = .black.opacity(0.1))
    padding(16.dp)
}

theme AppTheme {
    colors { primary = Color(0x5B21B6); surface = Color(0xFFFFFF) }
    typography {
        headline = Font("Inter", size = 24.sp, weight = .semibold)
        body = Font("Inter", size = 16.sp, weight = .regular)
    }
}
```

8. Platform Capability API

Orion detects the platform and bridges to native capabilities. Developers declare intent; the runtime resolves implementation.

8.1 Payments

```
@Capability(requires = Capability.Payments)
suspend fun checkout(cart: Cart): PaymentResult {
    val payment = Payment(cart.total, Currency.USD)
    // Apple -> Apple Pay | Android -> Google Pay | Fallback -> Stripe
    return Platform.payments.request(payment)
}
```

8.2 Notifications

```
@Capability(requires = Capability.Notifications)
suspend fun registerPush() {
    val token = Platform.notifications.register()
    // Apple -> APNs | Android -> FCM | Windows -> WNS
    api.registerDevice(token)
}
```

8.3 Biometrics

```
@Capability(requires = Capability.Biometrics)
suspend fun authenticate(): AuthResult {
    return Platform.biometrics.authenticate(
        reason = "Confirm your identity"
    )
    // Apple -> Face ID | Android -> BiometricPrompt | Win -> Hello
}
```

8.4 Platform Detection

```
when (Platform.current.os) {
    OS.MacOS   -> { /* WebKit */ }
    OS.Android -> { /* Blink */ }
    OS.Windows -> { /* Blink */ }
}
if (Platform.supports(Capability.Payments)) showPaymentButton()
```

9. Hardware Security Bridge

First-class HSM, secure enclave, and hardware token support. Private keys never leave the hardware.

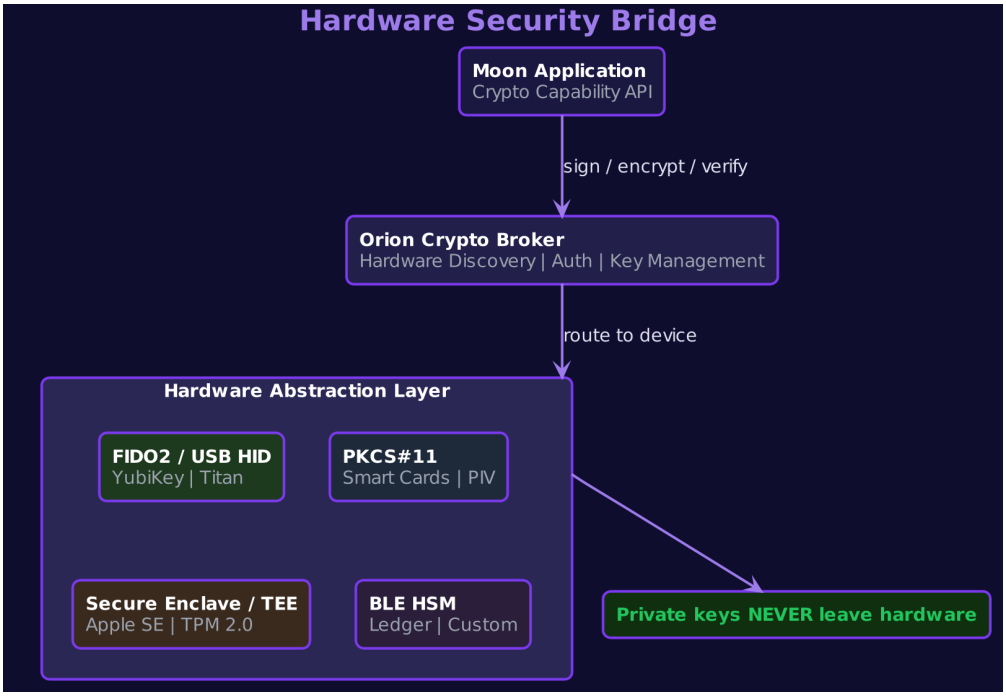

Figure 4: Hardware Security Bridge

9.1 Discovery & Operations

```
@Capability(requires = Capability.HardwareCrypto)
suspend fun signDocument(doc: ByteArray): Signature {
    val key = Platform.crypto.getKey(
        id = "signing-key", type = KeyType.Ed25519,
        requireHardware = true
    )
    return key.sign(doc)  // hardware signs, key never exposed
}
```

9.2 Encryption

```
suspend fun encryptMessage(
    msg: ByteArray, recipient: PublicKey
): EncryptedPayload {
    val key = Platform.crypto.getKey(
        id = "msg-key", type = KeyType.X25519, requireHardware = true
    )
    return key.encrypt(msg, recipient,
        EncryptionAlgorithm.XChaCha20Poly1305)
}
```

9.3 Capability Manifest

```
# aurora.manifest
crypto:
  operations: [sign, verify, encrypt, decrypt, keyExchange]
  key_types: [ed25519, x25519, ecdsa-p256]
  requires_hardware: true
  fallback: none
  attestation: required
  devices: [fido2, pkcs11, secure_enclave, bluetooth_hsm]
```

10. Package Manager (Aurora)

Aurora replaces npm, webpack, babel, eslint, and all config files with a single tool.

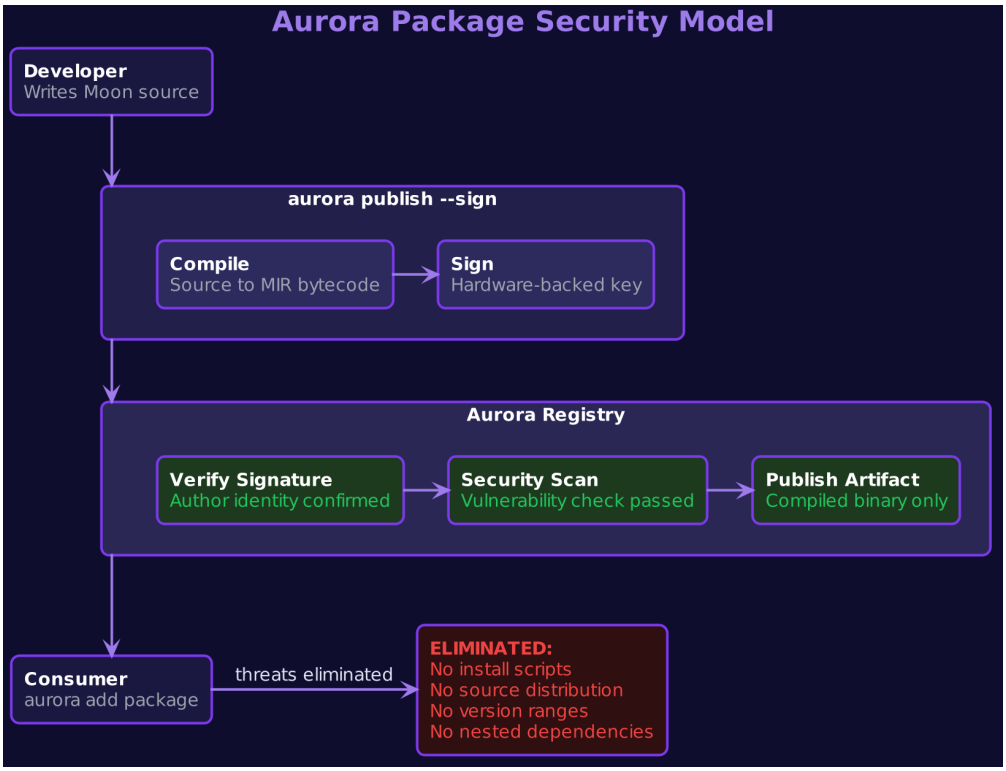

Figure 5: Aurora Security Model

10.1 CLI Commands

Command	Description
aurora init <name>	Create new project
aurora add <pkg>	Add dependency (exact pin)
aurora dev	Dev server + hot reload
aurora build --release	AOT compile
aurora test	Run tests
aurora publish --sign	Publish (requires signing key)
aurora audit --deep	Security audit

10.2 Manifest

```yaml
# aurora.manifest
name: my-app
version: 1.0.0
target: web
moon: 1.0

platform_capabilities:
  - payments
  - notifications
  - crypto: { requires_hardware: true }

dependencies:
  moon-ui: 2.1.0
  moon-http: 1.4.0
  moon-json: 3.0.0
```

10.3 Security Model

- All packages **cryptographically signed** by author
- Distributed as **compiled artifacts**, not source
- **No install scripts** during dependency resolution
- **Flat resolution** — no nested node_modules
- **Exact version pinning** — nothing changes silently
- **Registry-level scanning** for vulnerabilities

11. Standard Library

Comprehensive standard library. No third-party packages needed for typical web development.

Module	Purpose	Key Types
moon.core	Primitives, collections	List, Map, Set, Sequence
moon.http	HTTP client/server	HttpClient, Request, Response
moon.json	Serialization	JsonEncoder, JsonDecoder
moon.crypto	Cryptography	Hash, Cipher, Signature, Key
moon.time	Date, time, duration	Instant, Duration, TimeZone
moon.test	Testing framework	TestSuite, Assert, Mock
moon.concurrent	Concurrency	Flow, Actor, Channel, Mutex
moon.view	MoonUI framework	View, State, Navigator, Style
moon.platform	Platform caps	Payment, Notification, Biometric
moon.storage	Persistence	Database, KeyValue, FileSystem
moon.net	Networking	Socket, WebSocket, URL
moon.media	Audio/video/image	Camera, Microphone, MediaPlayer

12. Interoperability

Moon is a polyglot compilation target. Write in Moon directly or use supported source languages.

Language	Tier	Compilation Path
Moon (native)	Primary	Moon → MIR → Bytecode
Kotlin	Tier 1	Kotlin → KIR → MIR → Bytecode
Swift	Tier 1	Swift → SIL → MIR → Bytecode

Language	Tier	Compilation Path
Rust	Tier 2	Rust → LLVM IR → MIR → Bytecode
TypeScript	Migration	TS → AST → MIR (limited)

MIR (Moon Intermediate Representation) is the stable contract. Compiled packages are insulated from upstream language changes. Developers can write MIR directly as an escape hatch.

13. Compilation Pipeline

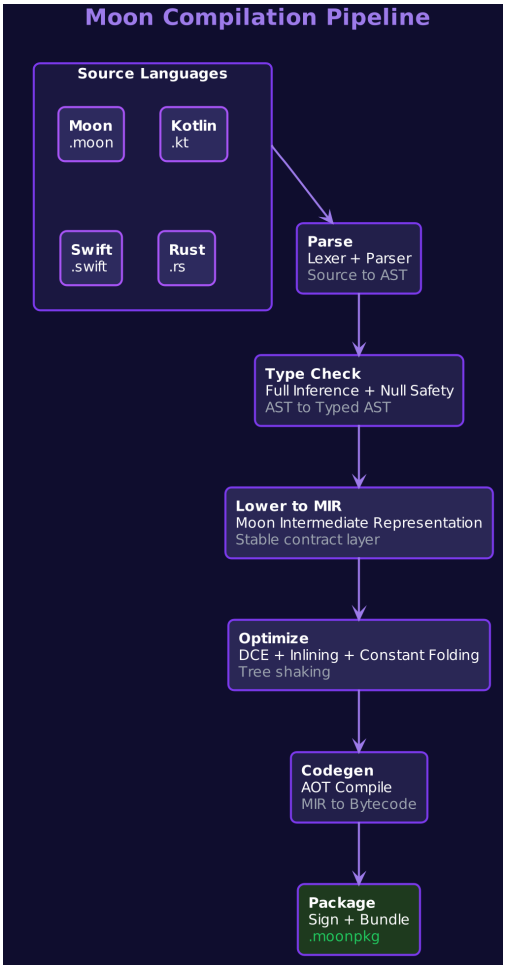

Figure 6: Moon Compilation Pipeline

13.1 Development Mode

```
aurora dev

# -> Moon Dev Server v1.0
# -> Watching src/ for changes
# -> http://localhost:3000
# -> Hot reload enabled
# -> [0.02s] Incremental recompile
# -> [0.01s] Hot reload applied (no page refresh)
```

13.2 Production Build

```
aurora build --release

# -> Compiling 47 source files...
# -> Type checking: passed
# -> Null safety: verified
# -> Capability validation: passed
# -> Tree shaking: removed 12 unused modules
# -> Output: dist/app.moonpkg (842 KB)
# -> Build complete in 2.3s
```

13.3 Stages

Stage	Input	Output	Description
Parse	.moon source	AST	Lexing and parsing
Type Check	AST	Typed AST	Inference + null safety
Lower	Typed AST	MIR	Moon IR generation
Optimize	MIR	Opt MIR	DCE, inlining, folding
Codegen	Opt MIR	Bytecode	Runtime bytecode
Package	Bytecode	moonpkg	Bundle + sign

14. Migration Strategy

14.1 Dual Runtime

Orion ships with both the legacy web engine (HTML/CSS/JS) and the Moon runtime. Legacy sites work unchanged. New apps target Moon. A bridge layer allows Moon apps to embed legacy content for gradual migration.

14.2 Timeline

Phase	Target	Milestone
Alpha	2026 Q3	Spec finalized, compiler prototype, dev tools
Beta	2027 Q1	Aurora registry, MoonUI beta, Orion Canary
Dev Preview	2027 Q3	Public SDK, documentation, example apps
Orion Integration	2028 Q1	Moon runtime in Orion stable (behind flag)
GA	2028 Q3	Flag removed, available to all Orion users
Legacy Deprecation	2030+	Begin JS engine deprecation signaling

14.3 TypeScript Migration

```
aurora migrate --from typescript --source ./src

# -> Analyzing 234 TypeScript files...
# -> Converting types: 98% automated
# -> Converting components to MoonUI: 85% automated
# -> Manual review needed: 12 files
# -> Output: ./src-moon/
```

15. Appendix: Grammar Reference

Abbreviated formal grammar for Moon in Extended Backus-Naur Form:

```
program = { declaration } ;
declaration = funDecl | viewDecl | classDecl | enumDecl
| interfaceDecl | actorDecl | typeAlias ;

funDecl = ["suspend"] "fun" id [typeParams]
"(" [params] ")" [":" type] block ;
viewDecl = "view" id "(" [params] ")" viewBlock ;
classDecl = ["data"|"sealed"] "class" id [typeParams]
["(" [params] ")"] [":" typeList] classBlock ;
actorDecl = "actor" id classBlock ;
enumDecl = "enum" "class" id enumBlock ;
navDecl = "navigation" id navBlock ;

type = id [typeArgs] ["?"] | funcType | tupleType ;
funcType = "(" [typeList] ")" "->" type ;

statement = valDecl | varDecl | assignment | expression
| ifExpr | whenExpr | forLoop | returnStmt ;
valDecl = "val" id [":" type] "=" expression ;
varDecl = ["weak"|"unowned"] "var" id [":" type] "=" expr ;
whenExpr = "when" "(" expr ")" "{" { whenEntry } "}" ;
lambda = "{" [params "->"] statements "}" ;
```

■■■ End of Specification ■■■

Moon Language • Codename Mars • Dark Matter Tech Confidential